Homework 3: Informed Search [105 points]

Instructions

In this assignment, you will explore a number of games and puzzles from the perspectives of informed and adversarial search.

A skeleton file homework3.py containing empty definitions for each question has been provided. Since portions of this assignment will be graded automatically, none of the names or function signatures in this file should be modified. However, you are free to introduce additional variables or functions if needed.

You may import definitions from any standard Python library, and are encouraged to do so in case you find yourself reinventing the wheel. If you are unsure where to start, consider taking a look at the data structures and functions defined in the `collections`, `itertools`, `queue`, and `random` modules.

You will find that in addition to a problem specification, most programming questions also include a pair of examples from the Python interpreter. These are meant to illustrate typical use cases, and should not be taken as comprehensive test suites.

Once you have completed the assignment, you should submit your file on Gradescope. You may submit as many times as you would like before the deadline, but only the last submission will be saved.

Note: Please do not run the autograder as a debugging tool. There is limited memory within the autograding software, so be mindful of how many times you submit.

0. Style [5 points]

Your code should follow the Python style guidelines set forth in PEP 8, which was written in part by the creator of Python. Our autograders will automatically scan your submission for style errors using the pycodestyle library on default settings. If your submission contains any style errors, the autograder will show you some of them and you will not receive these 5 points. You can use pycodestyle or any other tool you like to make sure that your submission conforms to PEP 8 guidelines.

1. Tile Puzzle [55 points]

Recall from class that the Eight Puzzle consists of a 3×3 board of sliding tiles with a single empty space. For each configuration, the only possible moves are to swap the empty tile with one of its neighboring tiles. The goal state for the puzzle consists of tiles 1-3 in the top row, tiles 4-6 in the middle row, and tiles 7 and 8 in the bottom row, with the empty space in the lower-right corner.

In this section, you will develop two solvers for a generalized version of the Eight Puzzle, in which the board can have any number of rows and columns. We have suggested an approach similar to the one used to create a Lights Out solver in Homework 2, and indeed, you may find that this pattern can be abstracted to cover a wide range of puzzles. If you wish to use the provided GUI for testing (described in more detail at the end of the section), then your implementation must adhere to the

recommended interface. However, this is not required, and no penalty will imposed for implementing a different approach.

A natural representation for this puzzle is a two-dimensional list of integer values between 0 and $r \cdot c - 1$ (inclusive), where r and c are the number of rows and columns in the board, respectively. In this problem, we will adhere to the convention that the 0-tile represents the empty space.

1. **[0 points]** Initialization

   In the `TilePuzzle` class, write an initialization method `__init__(self, board)` that stores an input board of this form described above for future use. You additionally may wish to store the dimensions of the board as separate internal variables, as well as the location of the empty tile.

2. **[0 points]** *Suggested infrastructure.*

   Get board: In the `TilePuzzle` class, write a method `get_board(self)` that returns the internal representation of the board stored during initialization.

```
>>> p = TilePuzzle([[1, 2], [3, 0]])
>>> p.get_board()
[[1, 2], [3, 0]]
```

```
>>> p = TilePuzzle([[0, 1], [3, 2]])
>>> p.get_board()
[[0, 1], [3, 2]]
```

   Create a TilePuzzle: Write a top-level function `create_tile_puzzle(rows, cols)` that returns a new `TilePuzzle` of the specified dimensions, initialized to the starting configuration. Tiles 1 through $r \cdot c - 1$ should be arranged starting from the top-left corner in row-major order, and tile 0 should be located in the lower-right corner.

```
>>> p = create_tile_puzzle(3, 3)
>>> p.get_board()
[[1, 2, 3], [4, 5, 6], [7, 8, 0]]
```

```
>>> p = create_tile_puzzle(2, 4)
>>> p.get_board()
[[1, 2, 3, 4], [5, 6, 7, 0]]
```

   Perform a move: In the `TilePuzzle` class, write a method `perform_move(self, direction)` that attempts to swap the empty tile with its neighbor in the indicated direction, where valid inputs are limited to the strings `"up"`, `"down"`, `"left"`, and `"right"`. If the given direction is invalid, or if the move cannot be performed, then no changes to the puzzle should be made. The method should return a Boolean value indicating whether the move was successful.

```
>>> p = create_tile_puzzle(3, 3)
>>> p.perform_move("up")
True
>>> p.get_board()
[[1, 2, 3], [4, 5, 0], [7, 8, 6]]
```

```
>>> p = create_tile_puzzle(3, 3)
>>> p.perform_move("down")
False
>>> p.get_board()
[[1, 2, 3], [4, 5, 6], [7, 8, 0]]
```

Scramble the puzzle: In the `TilePuzzle` class, write a method `scramble(self, num_moves)` which scrambles the puzzle by calling `perform_move(self, direction)` the indicated number of times, each time with a random direction. This method of scrambling guarantees that the resulting configuration will be solvable, which may not be true if the tiles are randomly permuted. *Hint:* The `random` module contains a function `random.choice(seq)` which returns a random element from its input sequence.

Check if solved: In the `TilePuzzle` class, write a method `is_solved(self)` that returns whether the board is in its starting configuration.

```
>>> p = TilePuzzle([[1, 2], [3, 0]])
>>> p.is_solved()
True
```

```
>>> p = TilePuzzle([[0, 1], [3, 2]])
>>> p.is_solved()
False
```

Create a copy: In the `TilePuzzle` class, write a method `copy(self)` that returns a new `TilePuzzle` object initialized with a **deep copy** of the current board. Changes made to the original puzzle should not be reflected in the copy, and vice versa.

```
>>> p = create_tile_puzzle(3, 3)
>>> p2 = p.copy()
>>> p.get_board() == p2.get_board()
True
```

```
>>> p = create_tile_puzzle(3, 3)
>>> p2 = p.copy()
>>> p.perform_move("left")
>>> p.get_board() == p2.get_board()
False
```

Generate successors: In the `TilePuzzle` class, write a method `successors(self)` that yields all successors of the puzzle as `(direction, new-puzzle)` tuples. The second element of each successor should be a new `TilePuzzle` object whose board is the result of applying the corresponding move to the current board. The successors may be generated in whichever order is most convenient, as long as successors corresponding to unsuccessful moves are not included in the output.

```
>>> p = create_tile_puzzle(3, 3)
>>> for move, new_p in p.successors():
...     print(move, new_p.get_board())
...
up   [[1, 2, 3], [4, 5, 0], [7, 8, 6]]
left [[1, 2, 3], [4, 5, 6], [7, 0, 8]]
```

```
>>> b = [[1,2,3], [4,0,5], [6,7,8]]
>>> p = TilePuzzle(b)
>>> for move, new_p in p.successors():
...     print(move, new_p.get_board())
...
up    [[1, 0, 3], [4, 2, 5], [6, 7, 8]]
down  [[1, 2, 3], [4, 7, 5], [6, 0, 8]]
left  [[1, 2, 3], [0, 4, 5], [6, 7, 8]]
right [[1, 2, 3], [4, 5, 0], [6, 7, 8]]
```

3. **[25 points]** In the `TilePuzzle` class, write a method `find_solutions_iddfs(self)` that yields all optimal solutions to the current board, represented as lists of moves. Valid moves include the four strings *`"up"`*, *`"down"`*, *`"left"`*, and *`"right"`*, where each move indicates a single swap of the empty tile with its neighbor in the indicated direction. Your solver should be implemented using an iterative deepening depth-first search (IDDFS), consisting of a series of depth-first searches limited at first to 0 moves, then 1 move, then 2 moves, and so on. You may assume that the board is solvable. The order in which the solutions are produced is unimportant, as long as all optimal solutions are present in the output.

   Hint: This method is most easily implemented using recursion. First define a recursive helper method `iddfs_helper(self, limit, moves)` that yields all solutions to the current board of length no more than `limit` which are continuations of the provided move list. Your main method will then call this helper function in a loop, increasing the depth limit by one at each iteration, until one or more solutions have been found. Note that this helper function should find all solutions within the step `limit` based on the `moves` already taken.

```
>>> b = [[4,1,2], [0,5,3], [7,8,6]]
>>> p = TilePuzzle(b)
>>> solutions = p.find_solutions_iddfs()
>>> next(solutions)
['up', 'right', 'right', 'down', 'down']
```

```
>>> b = [[1,2,3], [4,0,8], [7,6,5]]
>>> p = TilePuzzle(b)
>>> list(p.find_solutions_iddfs())
[['down', 'right', 'up', 'left', 'down',
 'right'], ['right', 'down', 'left',
 'up', 'right', 'down']]
```

4. **[30 points]** In the `TilePuzzle` class, write a method `find_solution_a_star(self)` that returns an optimal solution to the current board, represented as a list of direction strings. If multiple optimal solutions exist, any of them may be returned. Your solver should be implemented as an A^* search using the Manhattan distance heuristic, which is reviewed below. You may assume that the board is solvable. During your search, you should take care not to add positions to the queue that have already been visited. It is recommended that you use the `PriorityQueue` class from the `queue` module.

   Recall that the Manhattan distance between two locations (r_1, c_1) and (r_2, c_2) on a board is defined to be the sum of the componentwise distances: $|r_1 - r_2| + |c_1 - c_2|$. The Manhattan distance heuristic for an entire puzzle is then the sum of the Manhattan distances between each tile and its solved location.

   Hint: When comparing class objects, recall Python's magic methods from the Python module, and see more with this helpful resource (link).

```
>>> b = [[4,1,2], [0,5,3], [7,8,6]]
>>> p = TilePuzzle(b)
>>> p.find_solution_a_star()
['up', 'right', 'right', 'down', 'down']
```

```
>>> b = [[1,2,3], [4,0,5], [6,7,8]]
>>> p = TilePuzzle(b)
>>> p.find_solution_a_star()
['right', 'down', 'left', 'left', 'up',
 'right', 'down', 'right', 'up', 'left',
```

```
 'left', 'down', 'right', 'right']
```

5. Interactive GUI: If you implemented the suggested infrastructure described in this section, you can play with an interactive version of the Tile Puzzle using the provided GUI by running the following command:

```
python3 homework3_tile_puzzle_gui.py rows cols
```

The arguments `rows` and `cols` are positive integers designating the size of the puzzle.

In the GUI, you can use the arrow keys to perform moves on the puzzle, and can use the side menu to scramble or solve the puzzle. The GUI is merely a wrapper around your implementations of the relevant functions, and may therefore serve as a useful visual tool for debugging.

2. Grid Navigation [20 points]

In this section, you will investigate the problem of navigation on a two-dimensional grid with obstacles. The goal is to produce the shortest path between a provided pair of points, taking care to maneuver around the obstacles as needed. Path length is measured in Euclidean distance. Valid directions of movement include up, down, left, right, up-left, up-right, down-left, and down-right.

Your task is to write a function `find_path(start, goal, scene)` which returns the shortest path from the start point to the goal point that avoids traveling through the obstacles in the grid. For this problem, points will be represented as two-element tuples of the form (row, column), and scenes will be represented as two-dimensional lists of Boolean values, with `False` values corresponding empty spaces and `True` values corresponding to obstacles. Your output should be the list of points in the path, and should explicitly include both the start point and the goal point. Your implementation should consist of an A^* search using the straight-line Euclidean distance heuristic. If multiple optimal solutions exist, any of them may be returned. If no optimal solutions exist, or if the start point or goal point lies on an obstacle, you should return the sentinal value `None`.

```
>>> scene = [[False, False, False],
...          [False, True , False],
...          [False, False, False]]
>>> find_path((0, 0), (2, 1), scene)
[(0, 0), (1, 0), (2, 1)]
```

```
>>> scene = [[False, True, False],
...          [False, True, False],
...          [False, True, False]]
>>> print(find_path((0, 0), (0, 2), scene))
None
```

Once you have implemented your solution, you can visualize the paths it produces using the provided GUI by running the following command:

```
python3 homework3_grid_navigation_gui.py scene_path
```

The argument `scene_path` is a path to a scene file storing the layout of the target grid and obstacles. We use the following format for textual scene representation: "`.`" characters correspond to empty spaces, and "`X`" characters correspond to obstacles.

3. Linear Disk Movement, Revisited [20 points]

Recall the Linear Disk Movement section from Homework 2. The starting configuration of this puzzle is a row of ℓ cells, with disks located on cells 0 through $n-1$. The goal is to move the disks to the

end of the row using a constrained set of actions. At each step, a disk can only be moved to an adjacent empty cell, or to an empty cell two spaces away, provided another disk is located on the intervening square.

In a variant of the problem, the disks were distinct rather than identical, and the goal state was amended to stipulate that the final order of the disks should be the reverse of their initial order.

Implement an improved version of the `solve_distinct_disks(length, n)` function from Homework 2 that uses an A^* search rather than an uninformed breadth-first search to find an optimal solution. As before, the exact solution produced is not important so long as it is of minimal length. You should devise a heuristic which is admissible but informative enough to yield significant improvements in performance.

Hint: Recall that an admissable heuristic must not overestimate the distance to the goal. Review the textbook's example of relaxing the restrictions for tiles in the Eight Puzzle, and consider the rules of disk movement of this puzzle.

4. Feedback [5 points]

1. [**1 point**] Approximately how many hours did you spend on this assignment? Please enter your hours spent as a string.
2. [**2 points**] Which aspects of this assignment did you find most challenging? Were there any significant stumbling blocks?
3. [**2 points**] Which aspects of this assignment did you like? Is there anything you would have changed?